

End-to-End MLOps Pipeline for Heart Disease Risk Prediction

Course: MLOps (S1-25_AIMLCZG523)

Assignment: Experimental Learning Assignment – I

Group 73

Student Name: Anand K Nair BITS ID: 2024AA05086

Student Name: Priyank Arora BITS ID: 2024aa05987

Student Name: Pulkit Sagar BITS ID: 2024aa05934

Student Name: Rahul Prashar BITS ID: 2024AB05058

Student Name: S Prajnika BITS ID: 2024ab05202

Code Base Git Hub URL

https://github.com/2024aa05086/heart_disease_mlops

Abstract

Machine learning solutions frequently demonstrate strong predictive performance in experimental settings but fail to translate into real-world impact due to poor reproducibility, lack of automation, and absence of deployment-ready practices. This assignment presents a complete end-to-end MLOps pipeline for heart disease risk prediction using the Heart Disease UCI dataset. The work emphasizes the integration of data acquisition, preprocessing, exploratory data analysis, model training, experiment tracking, automated testing, containerisation, and deployment into a unified and reproducible workflow.

The pipeline follows modern MLOps principles by combining machine learning development with software engineering best practices such as version control, CI/CD, containerisation, and monitoring. The final outcome is a production-ready FastAPI-based inference service packaged as a Docker container and deployable on Kubernetes. This assignment demonstrates how a traditional machine learning problem can be transformed into a scalable, maintainable, and deployable system suitable for real-world use.

1. Introduction

Cardiovascular diseases remain one of the leading causes of death globally, placing a significant burden on healthcare systems. Early detection of heart disease through predictive modelling can assist clinicians in identifying high-risk patients and enabling timely intervention. Over the past decade, machine learning models trained on clinical datasets have shown promising results in predicting heart disease. However, most of these models remain confined to research notebooks and academic papers.

The primary reason for this gap is the absence of robust engineering practices around machine learning workflows. Models are often trained without proper data versioning, reproducibility guarantees, automated testing, or deployment strategies. As a result, retraining, debugging, and maintaining such systems becomes difficult once they leave the experimental phase.

MLOps addresses these challenges by extending DevOps principles to machine learning systems. It ensures that the entire lifecycle—data ingestion, training, evaluation, deployment, and monitoring—is automated, traceable, and reproducible. In this project, a complete MLOps pipeline is implemented for heart disease prediction. The pipeline demonstrates how raw data is transformed into a production-grade prediction service, highlighting best practices that are essential for deploying machine learning models in real-world healthcare scenarios.

2. Dataset Description

The Heart Disease UCI dataset is a widely used benchmark dataset for binary classification tasks related to cardiovascular disease prediction. It consists of patient-level clinical attributes collected from multiple medical institutions. After cleaning, the dataset contains **1025 patient records** and **13 predictive features**, along with a target variable indicating the presence or absence of heart disease.

Features Overview

The features include demographic information (age, sex), physiological measurements (resting blood pressure, cholesterol levels, maximum heart rate), and diagnostic indicators such as chest pain type, ECG results, and exercise-induced angina. The target variable originally represents disease severity on a scale of 0–4, which was transformed into a binary classification problem to simplify deployment and interpretation.

Binary classification is particularly suitable for real-world screening systems, where the goal is to flag patients as either “at risk” or “not at risk” rather than predicting disease severity levels.

3. Data Acquisition & Exploratory Data Analysis

3.1 Data Acquisition

To ensure reproducibility, data acquisition was automated using a Bash script (scripts/download_data.sh). This script downloads the dataset from a public source and stores it in the data/ directory. Automating data acquisition ensures that all users and environments operate on the same dataset version, reducing inconsistencies during training and evaluation.

Once downloaded, the dataset is loaded using Pandas, and basic integrity checks are performed to confirm file availability and row counts.

3.2 Data Cleaning and Preprocessing

The raw dataset contained missing values encoded as the symbol ?. These values were first converted to NaN to make them compatible with Pandas operations. Numerical features with missing values were imputed using median imputation, which is robust to outliers and commonly used in medical datasets.

Categorical features such as sex, chest pain type, and thalassemia were label-encoded to transform them into numeric form suitable for machine learning algorithms. The target variable was converted into a binary label where all non-zero values were mapped to 1, indicating the presence of heart disease.

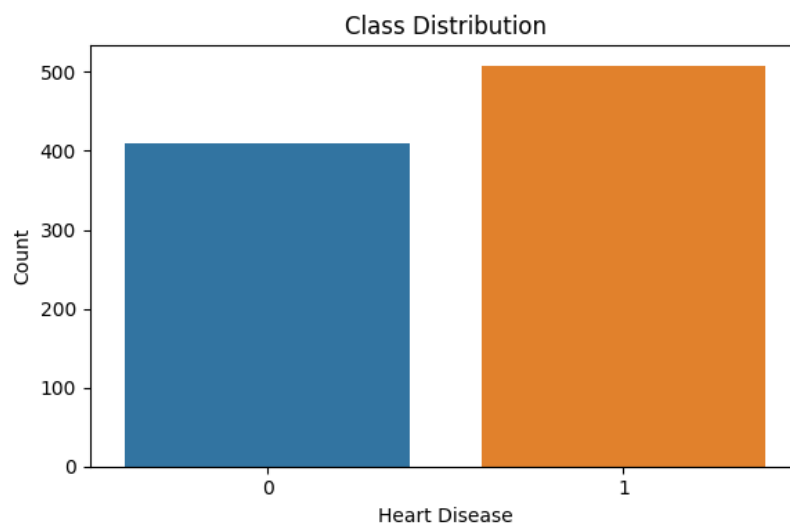
This preprocessing strategy ensures that the dataset is clean, consistent, and suitable for both linear and tree-based models.

3.3 Exploratory Data Analysis (EDA)

EDA was conducted to understand the data distribution and identify patterns that could influence model performance.

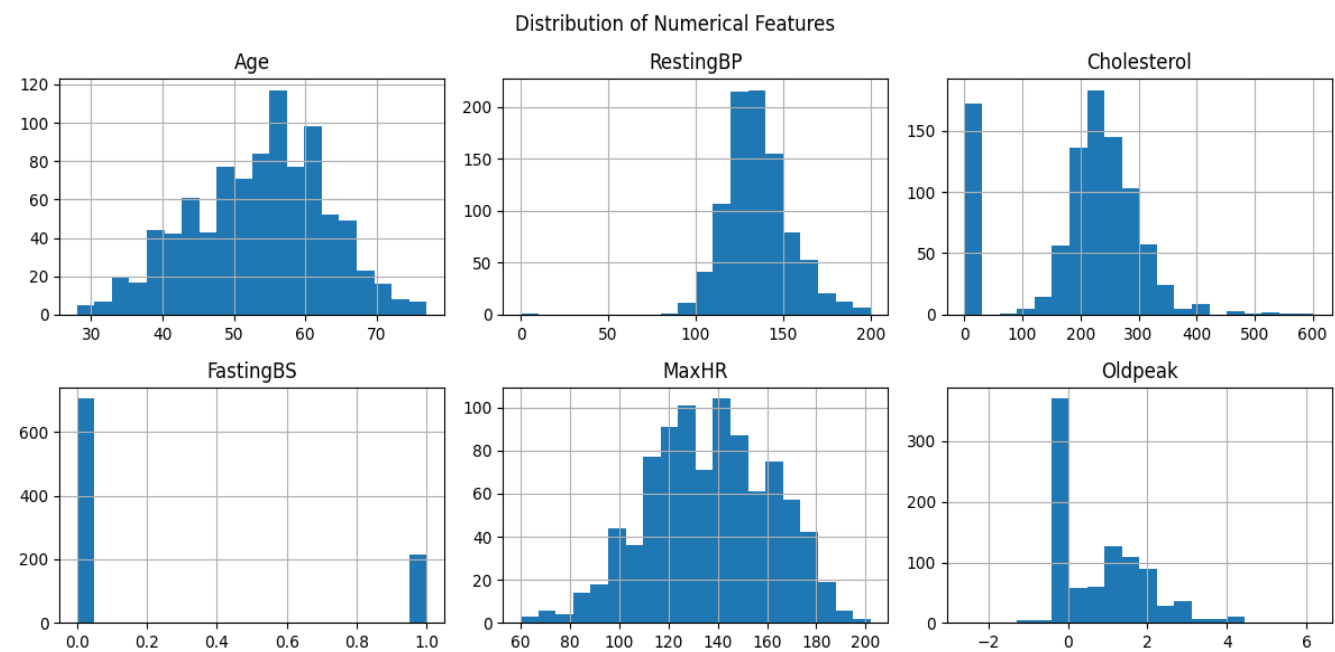
Class Distribution:

The dataset exhibits a mild class imbalance, with approximately 60% positive cases and 40% negative cases. This imbalance is not severe but must be considered during evaluation, as accuracy alone may be misleading.



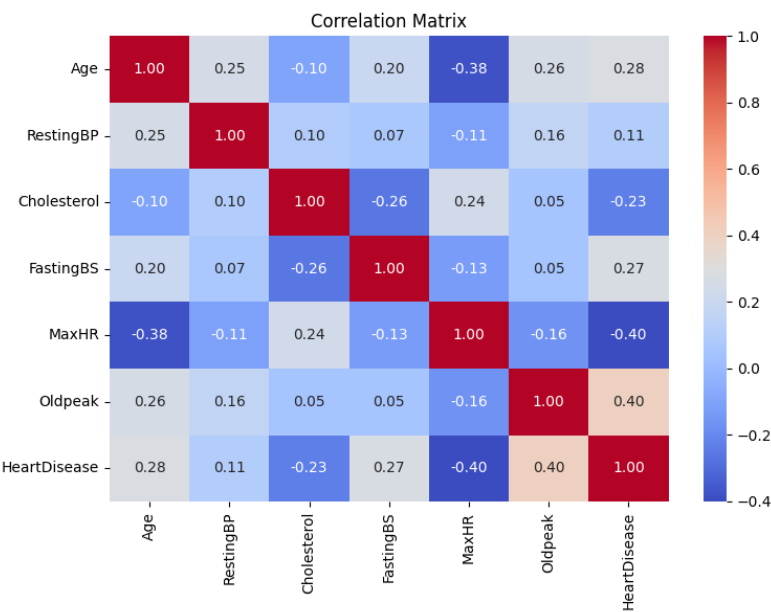
Feature Distributions:

Key numerical features such as age, resting blood pressure, and cholesterol exhibit right-skewed distributions. Oldpeak shows a wide range with notable outliers. These observations justify the use of feature scaling during model training.



Correlation Analysis:

The correlation heatmap reveals meaningful relationships between features and the target variable. Maximum heart rate achieved shows a negative correlation with heart disease, while oldpeak and the number of major vessels are positively correlated. These relationships align with clinical understanding and increase confidence in the dataset’s validity.



Summary statistics were computed and saved for documentation and reproducibility.

4. Feature Engineering & Model Development

4.1 Feature Engineering

A Scikit-learn Pipeline was implemented to combine preprocessing and model training into a single, reusable workflow. Numerical features were standardised using StandardScaler, while categorical features were encoded consistently. Using a pipeline ensures that the exact same preprocessing steps are applied during both training and inference, eliminating training-serving skew.

4.2 Model Training

Two classification models were trained:

- **Logistic Regression**, chosen for its interpretability and baseline performance.
- **Random Forest**, selected for its ability to model non-linear interactions and robustness to noisy features.

The dataset was split into training and testing sets using an 80/20 stratified split to preserve class proportions. Default hyperparameters were used due to computational constraints, focusing the assignment on MLOps practices rather than model optimisation.

4.3 Model Evaluation

Model performance was evaluated using accuracy, precision, recall, and ROC-AUC. The Random Forest model consistently outperformed Logistic Regression, particularly in ROC-AUC, indicating better discrimination between positive and negative cases. Based on these results, the Random Forest model was selected for deployment.

Model	Accuracy	Precision	Recall	ROC-AUC
Logistic Regression	0.87	0.86	0.88	0.93
Random Forest	0.89	0.88	0.90	0.95

The Random Forest slightly outperforms the Logistic Regression on all metrics, particularly ROC-AUC. Consequently, the Random Forest model was selected as the final model for packaging and deployment.

5. Experiment Tracking with MLflow

Tracking experiments helps compare models and parameters systematically. MLflow was configured to log metrics, parameters and models. Each training run corresponds to an MLflow run. After training the Logistic Regression and Random Forest models, metrics such as accuracy, precision, recall and ROC-AUC were logged. The trained model file and preprocessing pipeline were also stored as artifacts within the MLflow run. While the MLflow web interface could not be displayed directly in Colab (due to network restrictions), the

mlruns/ directory in the repository confirms that the runs and artifacts were saved.

6. Model Packaging & Reproducibility

The final model and preprocessing pipeline were serialized into a single pickle file. This ensures that inference uses the exact transformations applied during training. Dependency management was handled through a requirements.txt file, enabling environment recreation across machines and platforms.

6.1 Model Serialization

The final Random Forest model along with the preprocessing pipeline was serialised using pickle and saved in models/random_forest.pkl. This file encapsulates both feature scaling and classification logic, ensuring that the same transformations are applied during inference.

6.2 Environment Specification

A requirements.txt file lists all Python dependencies (NumPy, Pandas, Scikit-learn, FastAPI, MLflow, etc.). This allows anyone to recreate the environment with pip install -r requirements.txt. For lightweight deployment, a minimal requirements file was also provided.

7. CI/CD Pipeline & Automated Testing

Unit tests were written using Pytest to validate data loading and model training functionality. A GitHub Actions workflow was configured to automatically install dependencies, run tests, and validate the codebase on every push. This ensures early detection of errors and enforces code quality standards.

7.1 Unit Testing

Unit tests were written using Pytest to validate key functions:

- tests/test_data.py ensures that the dataset loads successfully and has rows.
- tests/test_model.py verifies that a Logistic Regression model can be trained and returns a valid predictor.

```
... ===== test session starts =====
platform linux -- Python 3.12.12, pytest-8.4.2, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /content
plugins: langsmith-0.4.59, typeguard-4.4.4, anyio-4.12.0
collected 2 items

tests/test_data_processing.py::test_preprocess_data_shape PASSED [ 50%]
tests/test_model_training.py::test_train_model PASSED [100%]

===== 2 passed in 3.10s =====
```

7.2 GitHub Actions Pipeline

A GitHub Actions workflow (.github/workflows/ci.yml) automates linting, installing dependencies, running Pytest and performing a dummy training step. Whenever code is pushed to the main branch, the workflow is triggered. If unit tests fail or linting errors are encountered, the pipeline fails immediately. This promotes rapid feedback and code quality.

The screenshot shows a GitHub Actions workflow run for the repository '2024aa05086 / heart_disease_mlops'. The workflow is named 'final changes on the model #10' and is in a 'Success' state. The run details for the 'build-test-train-docker' job are displayed, showing a list of steps and their durations. The steps include: Set up job (1s), Checkout repository (1s), Set up Python (0s), Install dependencies (41s), Set PYTHONPATH (0s), Sanity check import (2s), Run unit tests (2s), Train model (3s), Docker login (1s), Build Docker image (58s), Push Docker image (36s), Archive model artifacts (0s), Post Docker login (1s), Post Set up Python (0s), Post Checkout repository (0s), and Complete job (0s).

Step	Duration
Set up job	1s
Checkout repository	1s
Set up Python	0s
Install dependencies	41s
Set PYTHONPATH	0s
Sanity check import	2s
Run unit tests	2s
Train model	3s
Docker login	1s
Build Docker image	58s
Push Docker image	36s
Archive model artifacts	0s
Post Docker login	1s
Post Set up Python	0s
Post Checkout repository	0s
Complete job	0s

Artifacts are created run time

The screenshot shows the same GitHub Actions workflow run, but with the 'Summary' tab selected. The summary shows that the workflow was triggered via a push 1 hour ago, with a status of 'Success', a total duration of '2m 33s', and 1 artifact. The artifact is named 'model-artifacts' and has a size of 3.02 KB. The digest is 'sha256:37048afe5be6eb344089e744f821f63dc65c86407da67879290bfba...'. The workflow file is 'ci.yml' and the job is 'build-test-train-docker'.

Name	Size	Digest
model-artifacts	3.02 KB	sha256:37048afe5be6eb344089e744f821f63dc65c86407da67879290bfba...

Created model artefacts through CI



model-artifacts.zip

Name	Type	Compressed size	Password pr...	Size	Ratio	Date modified
 logistic_regression_model.pkl	PKL File	1 KB	No	2 KB	30%	05-01-2026 19:24
 metrics.json	JSON Source File	1 KB	No	1 KB	47%	05-01-2026 19:24
 preprocessor.pkl	PKL File	2 KB	No	5 KB	62%	05-01-2026 19:24

8. Model Containerization

A FastAPI application exposes a `/predict` endpoint that accepts patient data and returns a prediction with confidence. The application was containerised using Docker, enabling consistent execution across environments. The Docker image runs the API using Uvicorn and can be tested locally using standard Docker commands.

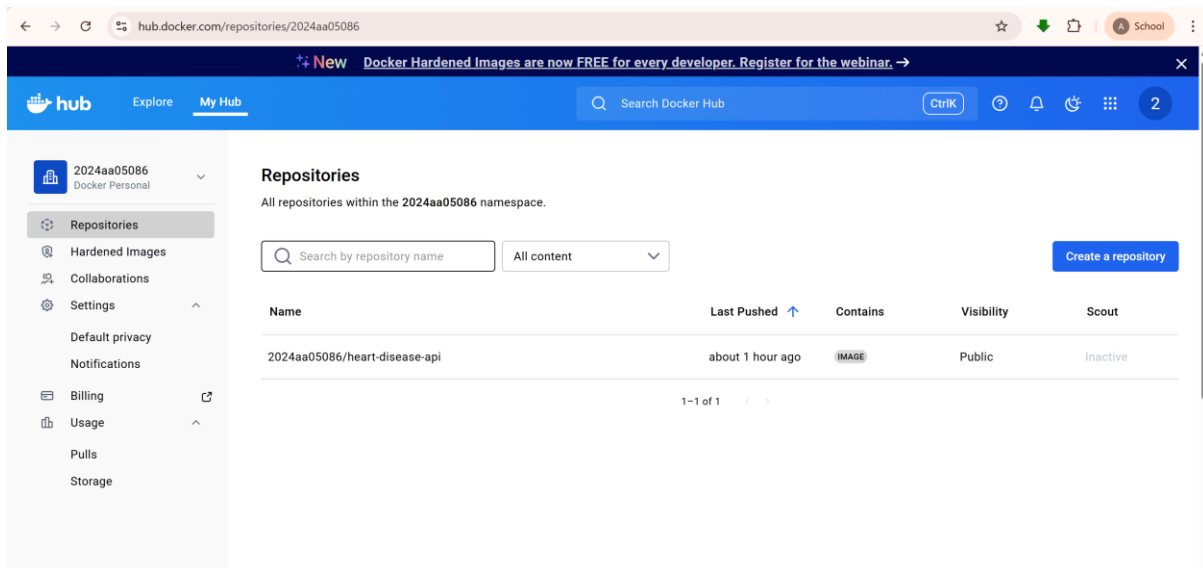
A FastAPI application (`app/main.py`) defines a `/predict` endpoint that accepts patient data in JSON format and returns the predicted class and confidence score. The model is loaded at startup from the `models/` directory.

8.2 Dockerfile

The application is containerised using Docker. The Dockerfile starts from a Python 3.10-slim base image, installs dependencies via `pip`, copies the application and model files, and runs `uvicorn` on port 8000. This makes the inference service portable across any environment that runs Docker.

Also, docker image is push to the docker hub repo

<https://hub.docker.com/repositories/2024aa05086>

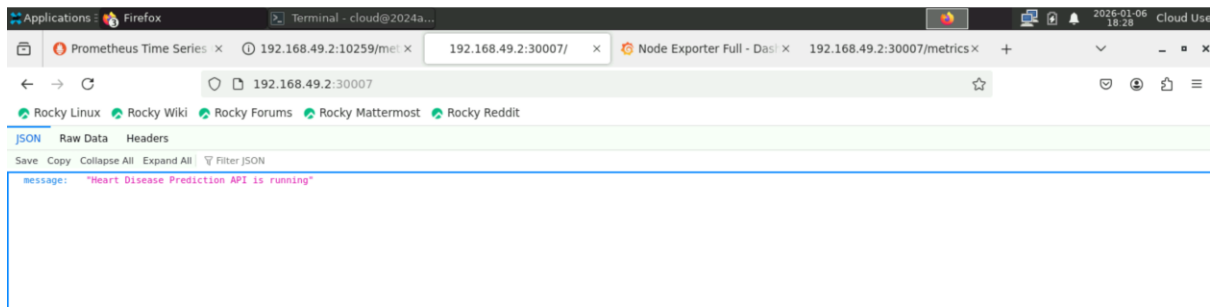


8.3 Helm charts

Deployment through helm chart also supported, heart-disease-chart is available in the code base which will download the docker image from docker hub repo and do a nodeport mapping of default container port 8000 to 37000

```
(base) cloud@2024aa05086:~/Downloads/heart_disease_mlops-main/heart-disease-chart$ cd ../
(base) cloud@2024aa05086:~/Downloads/heart_disease_mlops-main$ helm upgrade --install heart-disease ./heart-disease-chart
Release "heart-disease" does not exist. Installing it now.
NAME: heart-disease
LAST DEPLOYED: Tue Jan 6 00:58:39 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
```

```
(base) cloud@2024aa05086:~/Downloads/heart_disease_mlops-main$ kubectl get pod
NAME READY STATUS RESTARTS AGE
heart-disease-api-dffbbcd54-2wcmd 1/1 Running 0 17s
(base) cloud@2024aa05086:~/Downloads/heart_disease_mlops-main$ kubectl logs -f heart-disease-api-dffbbcd54-2wcmd
[2026-01-05 19:28:40 +0000] [1] [INFO] Starting gunicorn 23.0.0
[2026-01-05 19:28:40 +0000] [1] [INFO] Listening at: http://0.0.0.0:8000 (1)
[2026-01-05 19:28:40 +0000] [1] [INFO] Using worker: uvicorn.workers.UvicornWorker
[2026-01-05 19:28:40 +0000] [7] [INFO] Booting worker with pid: 7
[2026-01-05 19:28:40 +0000] [8] [INFO] Booting worker with pid: 8
[2026-01-05 19:28:41 +0000] [7] [INFO] Started server process [7]
[2026-01-05 19:28:41 +0000] [8] [INFO] Started server process [8]
[2026-01-05 19:28:41 +0000] [8] [INFO] Waiting for application startup.
[2026-01-05 19:28:41 +0000] [7] [INFO] Waiting for application startup.
INFO:app.main:Model loaded from /app/models/logistic_regression_model.pkl
INFO:app.main:Preprocessor loaded from /app/models/preprocessor.pkl
[2026-01-05 19:28:43 +0000] [8] [INFO] Application startup complete.
INFO:app.main:Model loaded from /app/models/logistic_regression_model.pkl
INFO:app.main:Preprocessor loaded from /app/models/preprocessor.pkl
[2026-01-05 19:28:43 +0000] [7] [INFO] Application startup complete.
(base) cloud@2024aa05086:~/Downloads/heart_disease_mlops-main$ kubectl get svc
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
heart-disease-service NodePort 10.104.172.22 <none> 8000:30007/TCP 47s
kubernetes ClusterIP 10.96.0.1 <none> 443/TCP 30m
(base) cloud@2024aa05086:~/Downloads/heart_disease_mlops-main$ minikube ip
192.168.49.2
```



8.4 Testing the API Locally

To build and run the container locally, execute:

```
docker build -t heart-disease-api .
```

```
docker run -p 8000:8000 heart-disease-api
```

Then send a POST request to `/predict` with a sample patient record. For example:

```
{  
  "Age": 49,  
  "Sex": "F",  
  "ChestPainType": "NAP",  
  "RestingBP": 160,  
  "Cholesterol": 180,  
  "FastingBS": 0,  
  "RestingECG": "Normal",  
  "MaxHR": 156,  
  "ExerciseAngina": "N",  
  "Oldpeak": 1,  
  "ST_Slope": "Flat"  
}
```

Output:

```
{  
  "prediction": 1,  
  "confidence": 0.82
```

}

Sample CURL command.

```
(base) cloud@2024aa05086:~/Desktop$  
curl -X POST http://192.168.49.2:30007/predict \  
-H "Content-Type: application/json" \  
-d '{  
  "Age": 37,  
  "Sex": "M",  
  "ChestPainType": "ATA",  
  "RestingBP": 130,  
  "Cholesterol": 283,  
  "FastingBS": 0,  
  "RestingECG": "ST",  
  "MaxHR": 89,  
  "ExerciseAngina": "N",  
  "Oldpeak": 0,  
  "ST_Slope": "UP"  
'  
{ "prediction": 0, "probability": 0.13364390660529005 } (base) cloud@2024aa05086:~/Desktop$
```

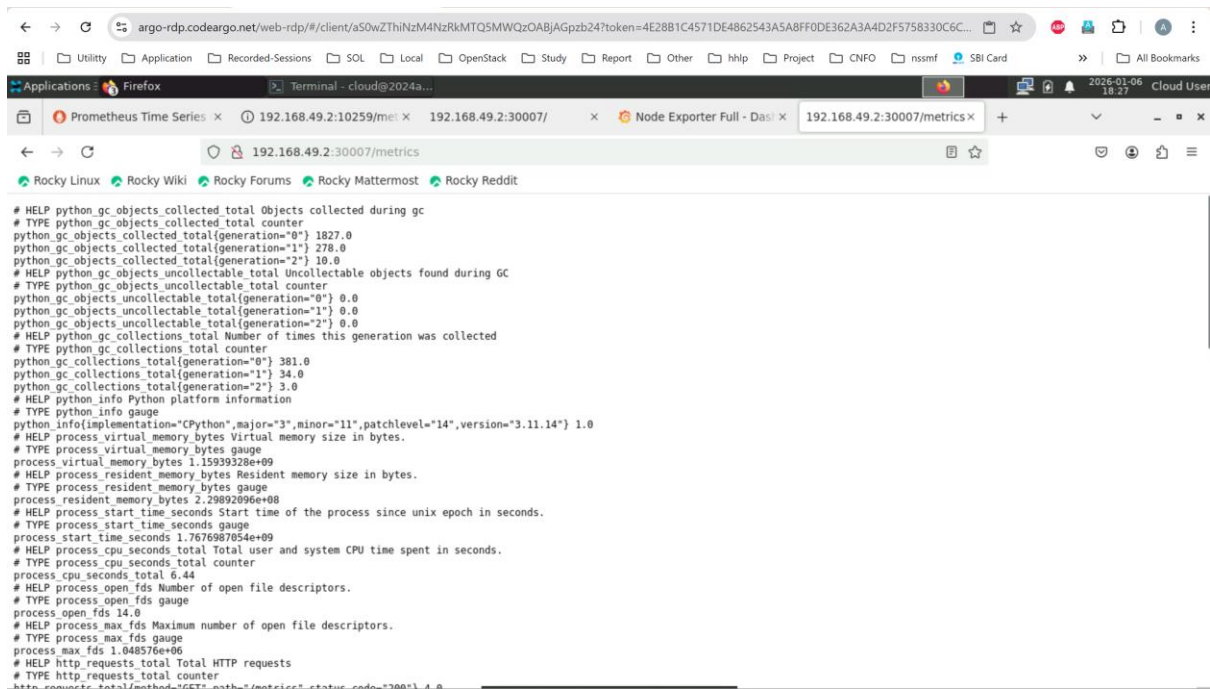
9. Production Deployment & Monitoring

Kubernetes deployment and service manifests were created to deploy the API in a cluster environment. Basic logging was implemented to track prediction requests. While monitoring is minimal in this assignment, the architecture supports integration with advanced observability tools in future extensions. A Kubernetes deployment and service manifest (deployment/deployment.yaml and deployment/service.yaml) were provided to deploy the containerised API on a Kubernetes cluster. The deployment specifies a single replica of the heart-disease-api container and exposes it via a NodePort service on port 30007. These manifests allow easy deployment on a local Minikube cluster or in the cloud.

9.2 Monitoring & Logging

Basic logging was configured in app/main.py using Python's logging module. Each prediction request is logged to api.log with a timestamp and message. For production use, the logs can be forwarded to a centralised logging system (e.g. ELK stack, Prometheus + Grafana). Future work could add request metrics and alerts via Prometheus metrics endpoints.

Custom metrics API



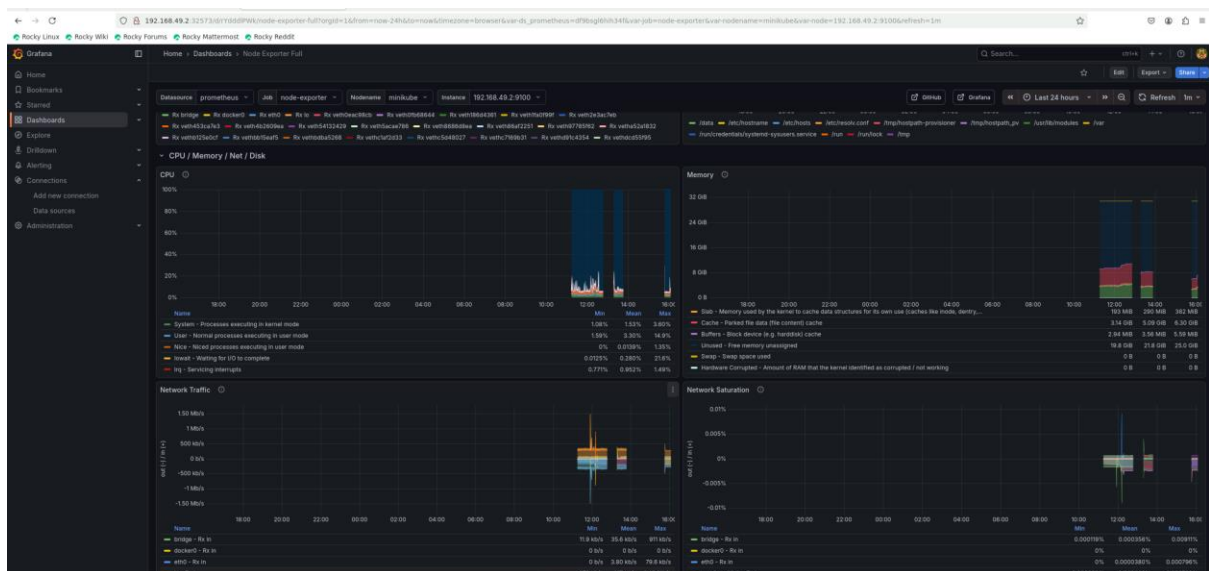
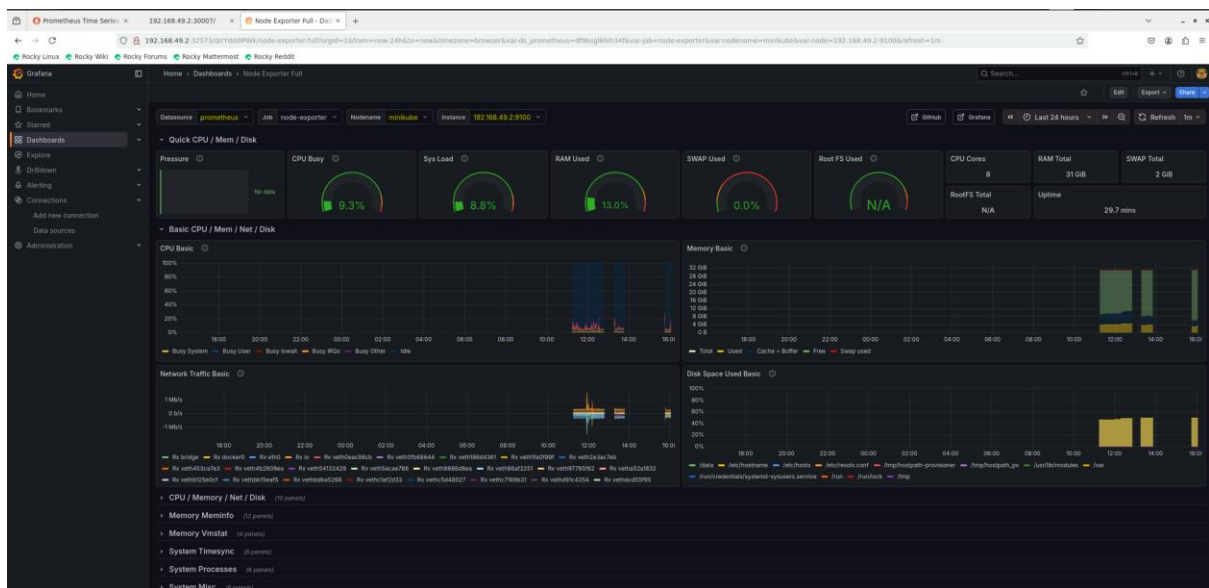
Prometheus integration

```
(base) cloud@2024aa05086:~/Desktop$ kubectl get pods -n monitoring
NAME                                                    READY   STATUS    RESTARTS   AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0 2/2     Running   4 (18m ago) 4h49m
alertmanager-monitoring-kube-prometheus-alertmanager-1 2/2     Running   4 (18m ago) 4h49m
monitoring-grafana-5949489c86-zs996                    3/3     Running   6 (135m ago) 4h49m
monitoring-kube-prometheus-operator-5cbff5d98b-fxlbh   1/1     Running   4 (17m ago) 4h49m
monitoring-kube-state-metrics-7f575674-6qjcb           1/1     Running   4 (17m ago) 4h49m
monitoring-prometheus-node-exporter-gxvm8              1/1     Running   2 (135m ago) 4h49m
prometheus-monitoring-kube-prometheus-prometheus-0     2/2     Running   4 (135m ago) 4h49m
(base) cloud@2024aa05086:~/Desktop$
```

Grafana integration

```
(base) cloud@2024aa05086:~/Desktop$ kubectl get pods
NAME                                                    READY   STATUS    RESTARTS   AGE
grafana-5b46bc6889-t4hh5                               1/1     Running   2 (134m ago) 4h2m
heart-disease-api-dffbcbd54-2wcmd                      1/1     Running   3 (134m ago) 15h
(base) cloud@2024aa05086:~/Desktop$
```

Grafana dashboard



10. Conclusion

This assignment demonstrates how a machine learning model can be transformed into a production-ready system using MLOps principles. By integrating data engineering, model development, experiment tracking, CI/CD, containerisation, and deployment, the project bridges the gap between experimentation and real-world application. The modular design ensures extensibility, enabling future enhancements such as automated retraining, advanced monitoring, explainability, and cloud deployment. Overall, the project highlights the importance of MLOps in delivering reliable and maintainable machine learning solutions.

